

BTLT-Individual-W05-02

Hướng dẫn:

1. Chạy chương trình:

- Chú thích:
 - Data gồm (MSSV, Count) được lưu ở **data.csv** trong project
 - Setting của chương trình được lưu ở **config.txt** trong project
- Tùy chọn của chương trình:
 - 1. Xem tất cả danh sách các Sinh Viên cần random để chọn
 - 2. Random ngẫu nhiên 1 Sinh Viên
 - Khác. Thoát chương trình
- Hình ảnh khi chạy chương trình:
 - Khi khởi động chương trình

```
Welcome to Random App!  
1. Print all random list  
2. Get random item from list  
Other. Exit!  
Please choose option:
```

- Xem tất cả Sinh Viên

```
Welcome to Random App!  
1. Print all random list  
2. Get random item from list  
Other. Exit!  
Please choose option: 1  
--- All Random List ---  
MSSV: 001      Count: 10      Probability: 21.6102%  
MSSV: 002      Count: 20      Probability: 17.3729%  
MSSV: 003      Count: 30      Probability: 13.1356%  
MSSV: 004      Count: 40      Probability: 8.8983%  
MSSV: 005      Count: 10      Probability: 21.6102%  
MSSV: 006      Count: 20      Probability: 17.3729%  
Press any key to continue . . . |
```

- Chọn ngẫu nhiên 1 Sinh Viên

```
Welcome to Random App!
1. Print all random list
2. Get random item from list
Other. Exit!
Please choose option: 2
MSSV: 001      Count: 10      Probability: 21.6102%
Press any key to continue . . . |
```

2. Thuật toán:

- Chọn 1 mốc (**flagCount**) dùng để so sánh các lượt phát biểu (**count**) với mốc (**flagCount**). Ta nhận thấy rằng số lượt phát biểu (**count**) càng xa mốc (**flagCount**) thì tỉ lệ được chọn càng cao vì các bạn phát biểu ít có nhiều cơ hội được chọn hơn.
- Từ ý tưởng trên, ta thấy tỉ lệ được chọn (**probability**) tỉ lệ với khoảng cách so với mốc (**flagCount**). Ngoại lệ là sẽ có các bạn phát biểu trên cả mốc (**flagCount**), lúc này chúng ta xem bạn như không cần được chọn với tỷ lệ **0%** và khoảng cách so với mốc bằng **0**. Bên cạnh đó, mặc định thì chương trình sẽ lấy lượt phát biểu nhiều nhất của Sinh Viên cộng với trung bình lượt phát biểu của Sinh Viên làm mốc.

$$\text{flagCount} = \text{maxCount} + \text{avgCount}$$

- Chương trình sẽ bắt đầu tính toán khoảng cách (**distance**) theo lượt phát biểu (**count**) và mốc (**flagCount**) và lưu lại vào từng Sinh Viên. Nếu khoảng cách (**distance**) âm thì quy về **0**

$$\text{distance} = \text{flagCount} - \text{count}$$

$$\text{distance} = \text{distance} > 0 ? \text{distance} : 0$$

- Chương trình tiếp tục tính xác suất theo công thức và lưu vào từng Sinh Viên. Trong đó **totalDistance** là tổng các khoảng cách của Sinh Viên

$$\text{probability} = (\text{distance} * 100.00) / \text{totalDistance}$$

- Chương trình chọn ngẫu nhiên Sinh Viên theo thuật toán dưới đây:
 - + Lúc này chúng ta có danh sách gồm (MSSV, probability (xác suất)). Tổng xác suất của tất cả sinh viên là 100.00%.
 - + Chúng ta tưởng tượng tổng xác suất của Sinh Viên là 1 đoạn thẳng **[0,100]**. Khi đó, Sinh Viên **A** có xác suất là 5.5% sẽ chiếm khoản **[0,5.5]** của đoạn thẳng. Kế

tiếp Sinh Viên **B** có xác suất là 10.00% sẽ chiếm khoản **[5.5, 15.5)**. Tương tự cho đến Sinh Viên cuối cùng.

- + Lúc này chúng ta chỉ cần random 1 con số từ 00 đến 100 và con số này thuộc khoản nào của Sinh Viên thì Sinh Viên ấy được chọn. Ví dụ ở trên nếu ta random được **4** thuộc **[0,5.5)** của Sinh Viên **A** thì Sinh Viên **A** được chọn. Nếu random được **12** thuộc **[5.5, 15.5)** của Sinh Viên **B** thì Sinh Viên **B** được chọn.
- + Thuật toán random ra số ngẫu nhiên từ 00 đến 100:

```
static float createRandomNumber() {  
    int min = 0, max = 10000; // 00.00 -> 100.00%  
    int res = rand() % (max - min + 1) + min;  
  
    return res / 100.00;  
}
```

- + Thuật toán tìm ra số ngẫu nhiên random được nằm trong khoản/đoạn nào

```
RandomItem* RandomApp::getRandomItem() {  
    float randomNumber = createRandomNumber();  
  
    for (int i = 0; i < _randomList.size(); i++) {  
        randomNumber -= _randomList[i]->getProbability();  
        if (randomNumber < 0) return _randomList[i];  
    }  
  
    return nullptr;  
}
```

- + Giải thích thêm cho thuật toán tìm số random được nằm trong khoản nào. Chúng ta lấy số random được trừ cho từng xác suất của Sinh Viên, nếu kết quả âm thì chúng ta được số random nằm trong khoản của Sinh Viên này. Ngược lại thì chúng ta tiếp tục trừ cho Sinh Viên tiếp theo. Ví dụ random được **12** ta biết là sẽ chọn Sinh Viên **B** như ví dụ ở trên, cùng xét theo thuật toán để làm rõ hơn.

- + Lần 1:

random = 12

result = random - probabilityA = 12 - 5.5 = 6.5 > 0

- + Lần 2:

random = 6.5

result = random - probabilityB = 6.5 - 10 = -3.5 < 0 (chọn)

- Hạn chế của thuật toán này là giá trị chọn mốc (**flagCount**), nếu chúng ta chọn giá trị mốc quá lớn, thì các khoản cách của lượt phát biểu và mốc cũng lớn theo, làm độ chênh lệch của các lượt phát biểu với nhau không nhiều. Vì thế xác suất của các Sinh Viên sẽ gần như nhau.

3. Điều chỉnh file config.txt:

- Như ở phần thuật toán chúng ta đã biết là chương trình của chúng ta sẽ sử dụng hàm **rand()** để tạo ra số ngẫu nhiên và chọn mốc (**flagCount**).
- Hàm **rand()** có thể tạo số khởi tạo cho thuật toán, vì thế trong file config.txt dòng đầu chúng ta chứa giá trị này. Nếu giá trị này **âm** thì chương trình sẽ lấy thời gian của máy làm giá trị khởi tạo (xài hàm **time(0)**).
- Dòng thứ 2 của file config.txt chúng ta chứa giá trị mốc (**flagCount**). Tương tự nếu giá trị này **âm** thì chương trình sẽ lấy mốc mặc định theo công thức ở trên (**flagCount = maxCount + avgCount**).